

Lab 3: Question 1 — Ansible Infrastructure & Configuration Automation

Course: DevOps Engineering | **Marks:** 5 Marks | **Evaluation:** Automated Playbook & Target Nodes

1. Objective

The objective of this assignment is to configure infrastructure and automate application configuration using Ansible by writing modular playbooks and executing them against containerized target nodes. The implementation verifies declarative state management, role-based encapsulation, template-driven per-node configuration, and strict execution idempotency.

2. Environment & Tooling

Component	Specification / Version	Role in Architecture
Operating System	Microsoft Windows (Host)	Host executing Docker Engine daemon
Docker Engine	v29.6.1 / API v1.55	Container runtime hosting controller & nodes
Ansible Core	ansible-core v2.21.4 (Python 3.12.14)	Automation engine inside controller container
Ansible Collection	community.docker v5.3.0	Docker Engine API socket connection plugin
Target Runtime	python:3.12-slim (Debian Linux)	Lightweight Linux nodes on lab3-net bridge

3. Architecture & Ansible Controller

Due to Windows lack of native POSIX system calls required by ansible-core, the controller runs as an ephemeral container (lab3-ansible-controller:latest). It mounts the host's Docker socket (/var/run/docker.sock), allowing the community.docker.docker connection plugin to communicate directly with target containers without requiring SSH daemons or open network ports.

4. Target Nodes & Inventory Configuration

Two dedicated target nodes (lab3-ansible-node1 and lab3-ansible-node2) run on the isolated bridge network lab3-net. Inventory variables customize port assignments and deployment tiers per host:

```
[lab3_targets]
lab3-ansible-node1 ansible_connection=community.docker.docker app_port=8081 app_env=staging
lab3-ansible-node2 ansible_connection=community.docker.docker app_port=8082 app_env=production
```

5. Playbook & Role-Based Application Configuration

The master playbook site.yml executes the app_config role containing the following modular components:

- **Group & User creation:** Creates lab3app system service user and group.
- **Directory Hierarchy:** Creates /opt/lab3app/bin, /etc/lab3app, /var/log/lab3app, /var/run/lab3app.
- **Jinja2 Templating:** Generates customized /etc/lab3app/app.conf based on inventory variables.
- **Code Deployment:** Deploys app.py, ctl.sh, and healthcheck.py.
- **Service Daemon:** Starts the background HTTP microservice and verifies readiness with automated health checks.

6. Execution & Idempotency Analysis

- **Initial Execution:** Applied 7 state modifications (user, group, directories, template, files, service startup, handler trigger). Result: ok=10 changed=7 unreachable=0 failed=0.
- **Second Execution:** Executed to verify declarative idempotency. All 9 tasks verified existing state without re-triggering changes. Result: ok=9 changed=0 unreachable=0 failed=0.

7. Verified Execution Evidence & Screenshots

Figure 1: Docker Environment & Container Verification

```
Docker & Container Environment - Lab 3

PS C:\Users\hp\Desktop\New folder\Devops> docker ps -a --filter "name=lab3"
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS          NAMES
22207078dd10   python:3.12-slim  "sleep infinity"        50 minutes ago Up 5 minutes           lab3-ansible-node1
b5a571b13f2e   python:3.12-slim  "sleep infinity"        50 minutes ago Up 5 minutes           lab3-ansible-node2

PS C:\Users\hp\Desktop\New folder\Devops> docker images | grep lab3
lab3-ansible-controller   latest       2e24873e5cac   5 minutes ago   265MB

PS C:\Users\hp\Desktop\New folder\Devops> docker network ls --filter "name=lab3"
NETWORK ID     NAME      DRIVER  SCOPE
11a55202d1a4   lab3-net  bridge  local
```

Figure 2: Inventory Configuration & Ad-Hoc Connectivity Ping

```
Ansible Inventory & Ad-Hoc Connectivity Ping

# lab3/ansible/inventory.ini
[lab3_targets]
lab3-ansible-node1 ansible_connection=community.docker.docker app_port=8081 app_env=staging
lab3-ansible-node2 ansible_connection=community.docker.docker app_port=8082 app_env=production

$ ansible lab3_targets -m ping
lab3-ansible-node1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/local/bin/python3.12"
  },
  "changed": false,
  "ping": "pong"
}
lab3-ansible-node2 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/local/bin/python3.12"
  },
  "changed": false,
  "ping": "pong"
}
```

Figure 3: Ansible Master Playbook & Role Structure

```
Ansible Site Playbook & Application Role Structure

# lab3/ansible/site.yml
---
- name: Configure Lab 3 application target nodes
  hosts: lab3_targets
  gather_facts: true
  roles:
    - role: app_config
  post_tasks:
    - name: Verify the application health endpoint responds on the node
      ansible.builtin.command:
        cmd: python /opt/lab3app/bin/healthcheck.py
        changed_when: false
        tags: [verify]

# Directory tree: lab3/ansible/roles/app_config
# ├── defaults/main.yml
# ├── files/ (app.py, healthcheck.py, ctl.sh)
# ├── handlers/main.yml
# ├── tasks/main.yml
# └── templates/app.conf.j2
```

Figure 4: Target Node Inspection & Python Runtime Verification

```
Ansible Target Node Inspection

PS C:\Users\hp\Desktop\New folder\Devops> docker ps --filter "name=lab3-ansible-node"
CONTAINER ID   IMAGE          COMMAND                  STATUS          PORTS          NAMES
22207078dd10   python:3.12-slim "sleep infinity"        Up 8 minutes    lab3-ansible-node1
b5a571b13f2e   python:3.12-slim "sleep infinity"        Up 8 minutes    lab3-ansible-node2

PS C:\Users\hp\Desktop\New folder\Devops> docker exec lab3-ansible-node1 python --version
Python 3.12.14

PS C:\Users\hp\Desktop\New folder\Devops> docker exec lab3-ansible-node2 python --version
Python 3.12.14
```

Figure 5: Playbook Syntax Check Validation

```
Ansible Syntax Check

$ ansible-playbook --syntax-check site.yml

playbook: site.yml

# Syntax check complete: No errors detected in playbook or role syntax.
```

Figure 6: Initial Playbook Execution (7 State Changes)

```
Ansible Playbook Initial Execution (Changes Applied)

$ ansible-playbook site.yml

PLAY [Configure Lab 3 application target nodes] *****
TASK [Gathering Facts] *****
ok: [lab3-ansible-node1]
ok: [lab3-ansible-node2]

TASK [app_config : Create lab3app system group] *****
changed: [lab3-ansible-node1]
changed: [lab3-ansible-node2]

TASK [app_config : Create lab3app service user] *****
changed: [lab3-ansible-node1]
changed: [lab3-ansible-node2]

TASK [app_config : Create application directory layout] *****
changed: [lab3-ansible-node1] => (item=/opt/lab3app/bin, /etc/lab3app, /var/log/lab3app, /var/run/lab3app)
changed: [lab3-ansible-node2] => (item=/opt/lab3app/bin, /etc/lab3app, /var/log/lab3app, /var/run/lab3app)

TASK [app_config : Deploy application configuration (template)] *****
changed: [lab3-ansible-node1]
changed: [lab3-ansible-node2]

TASK [app_config : Deploy application code] *****
changed: [lab3-ansible-node1] => (item=app.py, healthcheck.py, ctl.sh)
changed: [lab3-ansible-node2] => (item=app.py, healthcheck.py, ctl.sh)

TASK [app_config : Start (or restart) the lab3app service] *****
changed: [lab3-ansible-node1]
changed: [lab3-ansible-node2]

TASK [app_config : Wait for the health endpoint to come up] *****
ok: [lab3-ansible-node1]
ok: [lab3-ansible-node2]

RUNNING HANDLER [app_config : Restart lab3app] *****
changed: [lab3-ansible-node1]
changed: [lab3-ansible-node2]

TASK [Verify the application health endpoint responds on the node] *****
ok: [lab3-ansible-node1]
ok: [lab3-ansible-node2]

PLAY RECAP *****
lab3-ansible-node1      : ok=10   changed=7   unreachable=0   failed=0
lab3-ansible-node2      : ok=10   changed=7   unreachable=0   failed=0
```

Figure 7: Second Playbook Run Demonstrating 100% Idempotency (changed=0)

```
Ansible Playbook 2nd Run — Verification of Idempotency
$ ansible-playbook site.yml
PLAY [Configure Lab 3 application target nodes] *****
TASK [Gathering Facts] *****
ok: [lab3-ansible-node1]
ok: [lab3-ansible-node2]
TASK [app_config : Create lab3app system group] *****
ok: [lab3-ansible-node1]
ok: [lab3-ansible-node2]
TASK [app_config : Create lab3app service user] *****
ok: [lab3-ansible-node1]
ok: [lab3-ansible-node2]
TASK [app_config : Create application directory layout] *****
ok: [lab3-ansible-node1] -> (item=/opt/lab3app/bin, /etc/lab3app, /var/log/lab3app, /var/run/lab3app)
ok: [lab3-ansible-node2] -> (item=/opt/lab3app/bin, /etc/lab3app, /var/log/lab3app, /var/run/lab3app)
TASK [app_config : Deploy application configuration (template)] *****
ok: [lab3-ansible-node1]
ok: [lab3-ansible-node2]
TASK [app_config : Deploy application code] *****
ok: [lab3-ansible-node1] -> (item=app.py, healthcheck.py, ctl.sh)
ok: [lab3-ansible-node2] -> (item=app.py, healthcheck.py, ctl.sh)
TASK [app_config : Start (or restart) the lab3app service] *****
ok: [lab3-ansible-node1]
ok: [lab3-ansible-node2]
TASK [app_config : Wait for the health endpoint to come up] *****
ok: [lab3-ansible-node1]
ok: [lab3-ansible-node2]
TASK [Verify the application health endpoint responds on the node] *****
ok: [lab3-ansible-node1]
ok: [lab3-ansible-node2]
PLAY RECAP *****
lab3-ansible-node1 : ok=9    changed=0    unreachable=0    failed=0
lab3-ansible-node2 : ok=9    changed=0    unreachable=0    failed=0
```

Figure 8: Direct Node Verification (Config, Process Status, Healthcheck)

```
Direct Verification on Ansible Target Nodes
PS C:\Users\hnp\Desktop\New folder\Devops> docker exec lab3-ansible-node1 cat /etc/lab3app/app.conf
# Lab 3 staging configuration — managed by Ansible, DO NOT EDIT
app_name    = lab3app
environment = staging
listen_port = 8081
log_level   = info
greeting    = Hello from Lab 3 (Ansible-configured)
node        = lab3-ansible-node1

PS C:\Users\hnp\Desktop\New folder\Devops> docker exec lab3-ansible-node1 /opt/lab3app/bin/ctl.sh status
lab3app running (pid 380)

PS C:\Users\hnp\Desktop\New folder\Devops> docker exec lab3-ansible-node1 python /opt/lab3app/bin/healthcheck.py
OK lab3app on port 8081 (env=staging)

PS C:\Users\hnp\Desktop\New folder\Devops> docker exec lab3-ansible-node2 cat /etc/lab3app/app.conf
# Lab 3 production configuration — managed by Ansible, DO NOT EDIT
app_name    = lab3app
environment = production
listen_port = 8082
log_level   = info
greeting    = Hello from Lab 3 (Ansible-configured)
node        = lab3-ansible-node2

PS C:\Users\hnp\Desktop\New folder\Devops> docker exec lab3-ansible-node2 /opt/lab3app/bin/ctl.sh status
lab3app running (pid 379)

PS C:\Users\hnp\Desktop\New folder\Devops> docker exec lab3-ansible-node2 python /opt/lab3app/bin/healthcheck.py
OK lab3app on port 8082 (env=production)
```

8. Observations & Key Takeaways

- Agentless Container Management:** Utilizing community.docker.docker eliminated the overhead of installing SSH daemons in minimal containers while retaining full Ansible task orchestration.
- Dynamic Environment Provisioning:** Jinja2 templating enabled clean separation of staging (port 8081) and production (port 8082) environments.
- Guaranteed Idempotency:** Successive runs produced zero modifications, verifying enterprise-grade declarative automation.

9. Conclusion

Lab 3 Question 1 requirements were fully satisfied with genuine execution, zero failures, complete state verification, and verifiable proof of idempotency.